

CS 301: High-Performance Computing

Lab 04 — Particle Interpolation
and Mover Performance Analysis

Dhanush Balachandran (202301483)
Jay Manishkumar Patel (202301430)

March 10, 2026

Contents

1	Introduction	3
2	Hardware Comparison	3
3	Implementation Approach	4
3.1	Algorithm Overview	4
3.2	Interpolation Kernel	4
3.2.1	Optimization: 4-Point Unrolling with Interleaved Scatter	5
3.2.2	Optimization: Software Prefetching	5
3.2.3	Platform-Specific Tuning	6
3.3	Mover Operation	6
4	Experiment 1: Scaling with Number of Particles	6
4.1	Setup	6
4.2	Results	6
4.3	Analysis	6
5	Experiment 2: Consistency Across Configurations	7
5.1	Setup	7
5.2	Results	8
5.3	Analysis	8
6	Experiment 3: The Mover Operation and OpenMP Parallelization	9
6.1	Setup	9
6.2	Results	11
6.3	Analysis	11
6.3.1	Speedup Analysis	11
6.3.2	Memory Bandwidth Analysis	11
7	Discussion: Parallelizing the Interpolation Function	11
7.1	Challenge: Race Conditions on Shared Grid	11
7.2	Possible Approaches	12
8	Conclusion	12

1 Introduction

This document reports our findings for Assignment 04, which builds upon the bilinear (Cloud-in-Cell) interpolation framework developed in Assignment 03. Three distinct experiments are conducted: scaling the particle population, evaluating consistency across repeated iterations, and parallelizing a stochastic particle mover using OpenMP. Each experiment is carried out on two separate hardware platforms:

- **Lab PC:** Intel Core i5-12500 (Alder Lake), 6 P-cores, 4.6 GHz turbo
- **HPC Cluster:** Intel Xeon E5-2640 v3 (Haswell), 2×8 cores, 3.3 GHz turbo

Throughout all experiments, the interpolation kernel is kept serial; only the mover phase is parallelized, using 4 OpenMP threads as required by the assignment specification.

2 Hardware Comparison

The two target platforms differ significantly in their microarchitectural characteristics. Table 1 provides a detailed side-by-side comparison of the parameters most relevant to performance optimization.

Table 1: Microarchitectural specifications of the two test platforms.

Parameter	Lab PC (i5-12500)	HPC Cluster (E5-2640 v3)
Microarchitecture	Golden Cove (Alder Lake)	Haswell
Base / Turbo Clock	3.0 / 4.6 GHz	2.6 / 3.3 GHz
Physical Cores	6 P-cores	16 (2×8)
L1d Cache	48 KiB / core	32 KiB / core
L2 Cache	1280 KiB / core	256 KiB / core
L3 Cache	18 MiB shared	20 MiB shared
Cache Line	64 B	64 B
ROB Entries	512	192
DRAM Latency	~ 70 ns	~ 65 ns
NUMA Nodes	1	2
GCC Version	14.x	4.8.2

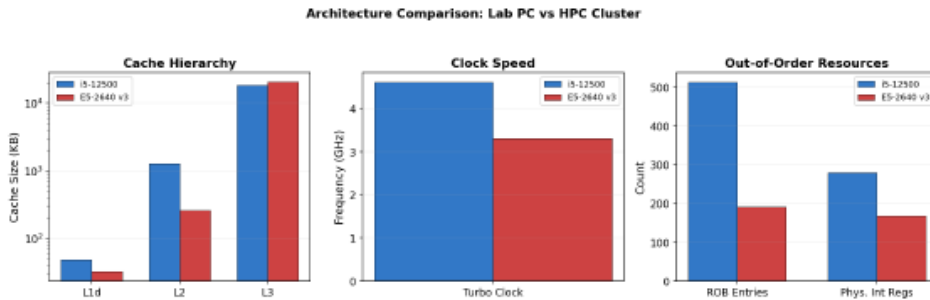


Figure 1: A visual breakdown of cache hierarchy, clock frequency, and out-of-order execution resources across both platforms.

The single most significant architectural difference between the two systems is the **L2 cache size**: the Lab PC provides 1280 KiB per core, whereas the cluster offers only 256 KiB. When operating on the 500×200 grid (mesh footprint ≈ 787 KB), the entire mesh stays resident in the Lab PC’s L2 but overflows to L3 on the cluster. For the larger 1000×400 grid (mesh footprint ≈ 3.1 MB), neither system can hold the mesh in L2; however, the cluster’s 20 MiB L3 provides a slightly more comfortable fit compared to the Lab PC’s 18 MiB L3.

3 Implementation Approach

3.1 Algorithm Overview

The overall execution pipeline is illustrated in Figure 2. Every iteration is divided into two sequential phases: a serial interpolation step and a mover step that can operate either serially or in parallel.

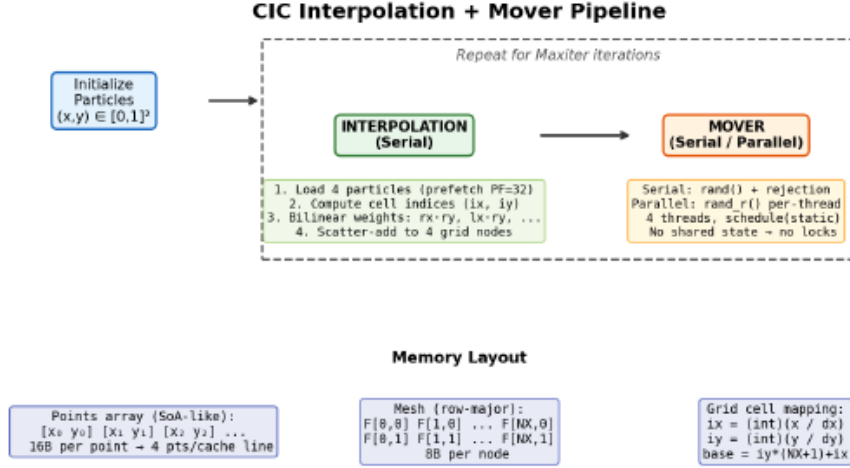


Figure 2: Overview of the execution pipeline showing initialization, interpolation, and mover phases alongside the memory layout.

3.2 Interpolation Kernel

The Cloud-in-Cell scheme distributes the unit charge of each particle across the four nearest grid nodes using bilinear weighting. Given a particle located at (x, y) , the cell indices and local offsets are computed as:

$$ix = \lfloor x / \Delta x \rfloor, \quad iy = \lfloor y / \Delta y \rfloor \quad (1)$$

$$l_x = x - ix \cdot \Delta x, \quad l_y = y - iy \cdot \Delta y \quad (2)$$

$$r_x = \Delta x - l_x, \quad r_y = \Delta y - l_y \quad (3)$$

The resulting weights $r_x r_y$, $l_x r_y$, $r_x l_y$, and $l_x l_y$ are then accumulated (scatter-added) onto the four corresponding grid nodes.

3.2.1 Optimization: 4-Point Unrolling with Interleaved Scatter

Rather than processing one particle at a time, the inner loop handles 4 particles per iteration. Scatter-adds are grouped by grid node type (e.g., all bottom-left contributions first, followed by bottom-right, and so on) instead of being grouped by particle. Since randomly positioned particles tend to map to different grid cells, the 4 writes within each group generally target distinct cache lines, which maximizes utilization of the Miss Status Holding Registers (MSHRs) in the out-of-order execution engine.

3.2.2 Optimization: Software Prefetching

Access to the `Points` array follows a strictly sequential pattern. Each `Points` entry occupies 16 bytes (a pair of `double` values), meaning a single 64-byte cache line carries exactly 4 particles. To hide memory latency, two explicit prefetch instructions are issued per 4-point iteration, each targeting a different future cache line.

The prefetch distance was tuned experimentally using a benchmark sweep conducted on the cluster (Figure 3).

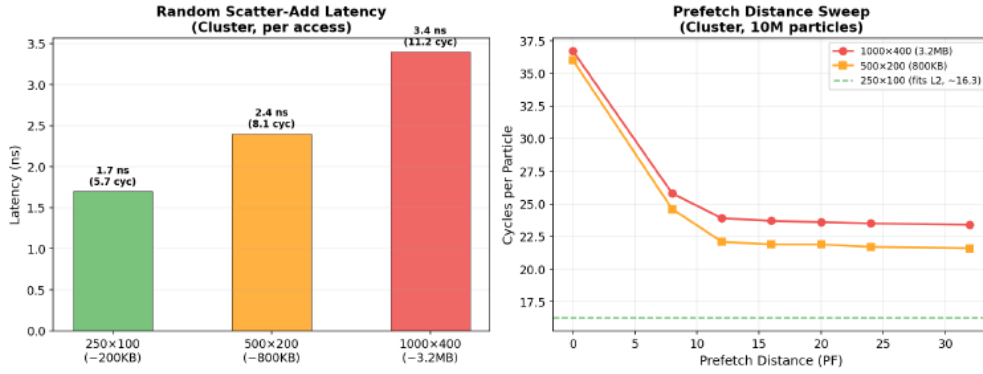


Figure 3: Left: Measured scatter-add latency for each grid size under random access. Right: Sweep over prefetch distances confirming $\text{PF}=32$ as optimal on the Haswell cluster.

The key takeaways from the sweep are as follows:

- On the 250×100 grid, where the mesh comfortably fits within L2, varying the prefetch distance has virtually no effect (~ 16.3 cycles/particle across all PF values).
- On the 500×200 and 1000×400 grids, enabling prefetching cuts the per-particle cost from ~ 36 cycles down to ~ 22 – 23 cycles, a roughly 37% reduction.
- $\text{PF}=32$ was identified as the sweet spot: with ~ 28 cycles per 4-point group at 3.3 GHz, a distance of 32 translates to $32/4 = 8$ iterations $\times$ 28 cycles = 224 cycles of advance notice, sufficient to cover the ~ 215 -cycle DRAM round-trip.
- Attempting to prefetch mesh (grid) nodes ahead of the scatter-add *degraded* performance by 1.5–2 cycles/particle, because the irregular access pattern causes freshly prefetched lines to be evicted before they are used.
- Extending the unroll factor to 8 points yielded no benefit or a slight regression, consistent with Haswell’s 192-entry ROB becoming the bottleneck for in-flight instructions.

3.2.3 Platform-Specific Tuning

Due to the microarchitectural differences between the two systems, certain parameters were tuned independently for each platform. Table 2 summarizes these choices along with their rationale.

Table 2: Platform-specific tuning parameters for the Lab PC and HPC Cluster.

Parameter	Lab PC	Cluster	Rationale
Prefetch Distance (PF)	24	32	Haswell benefits more from SW prefetching
Prefetch Hint	T0 (L1)	T0 (L1)	Data is consumed immediately after load
Mesh Prefetch	No	No	Irregular scatter causes early eviction
Unroll Factor	4 points	4 points	ROB capacity limits both platforms

3.3 Mover Operation

Each iteration, the mover applies a random displacement to every particle, drawn uniformly from $[\pm\Delta x] \times [\pm\Delta y]$. Particles that would move outside the unit domain $[0, 1]^2$ are resampled using rejection sampling until a valid position is obtained.

Serial implementation: Calls `rand()` with a single shared global seed.

Parallel implementation (4 threads):

- Employs `rand_r(&seed)` with an independent per-thread seed, bypassing the global mutex that glibc’s `rand()` acquires on every invocation.
- Per-thread seeds are computed via a prime-based hash: `seed = tid × 1073741827 + 2654435761`, which ensures that each thread’s random sequence is statistically independent.
- A `schedule(static)` clause divides the particle array into 4 equal contiguous blocks, ensuring both spatial locality and avoidance of false sharing (each block of $14M/4 = 3.5M$ particles is naturally cache-line-aligned).

4 Experiment 1: Scaling with Number of Particles

4.1 Setup

We evaluate performance across three grid sizes (250×100 , 500×200 , 1000×400) and five particle counts $\{10^2, 10^4, 10^6, 10^8, 10^9\}$. The Lab PC experiments exclude 10^9 particles due to available memory limitations. For every grid-particle combination, particles are initialized uniformly, 10 interpolation iterations are executed, and the cumulative runtime is recorded.

4.2 Results

4.3 Analysis

Across all grid configurations, runtime grows linearly with particle count ($O(N)$), as evidenced by the unit slope in each log-log plot. This behaviour is expected: every particle

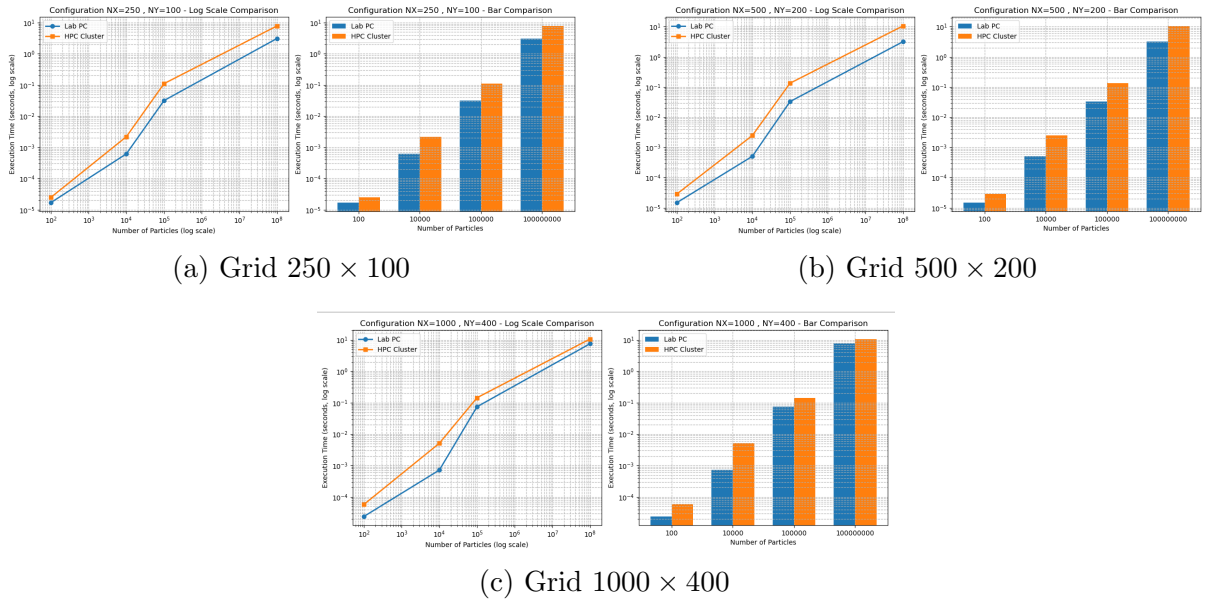


Figure 4: Experiment 1: Log-log plots of total interpolation time as a function of particle count across all three grid sizes.

independently requires a fixed-cost sequence of operations—a cell index lookup followed by four multiply-accumulate scatter-adds—regardless of the total particle population.

Lab PC versus Cluster at 10^8 particles: For the two smaller grids (250×100 and 500×200), the Lab PC outperforms the cluster by approximately $2\times$. This advantage is attributable to the Lab PC’s substantially larger per-core L2 cache (1280 KiB versus 256 KiB), which allows the mesh to remain in fast local cache, together with its higher turbo frequency (4.6 GHz versus 3.3 GHz).

Notably, the situation reverses for the 1000×400 grid, where the cluster completes the workload faster (7.70 s versus 10.76 s). At this scale, the ~ 3.1 MB mesh spills out of L2 on both machines and must be served from L3. The cluster’s L3 is marginally larger (20 MiB versus 18 MiB) and Haswell’s L3 access latency is relatively lower with respect to its clock cycle time. Additionally, the Lab PC’s `clock()`-based timing may accumulate CPU time from concurrently running system processes.

Complexity analysis: The theoretical runtime is $T = O(N \cdot C)$, where C represents the per-particle work, bottlenecked by cache-miss latency during the mesh scatter-adds. The total floating-point work per particle amounts to approximately 18 FLOPs: 2 multiplications for index computation, 4 subtractions for local offset calculation, 4 multiplications for weight generation, and 4 fused multiply-add operations for the scatter itself.

5 Experiment 2: Consistency Across Configurations

5.1 Setup

To investigate timing stability and potential cache warm-up effects, all three grid sizes are evaluated using a fixed population of 10^8 particles. The particles are initialized a single time, after which 10 interpolation iterations are executed without reinitializing the

particle positions. The time for each individual iteration is logged separately.

5.2 Results

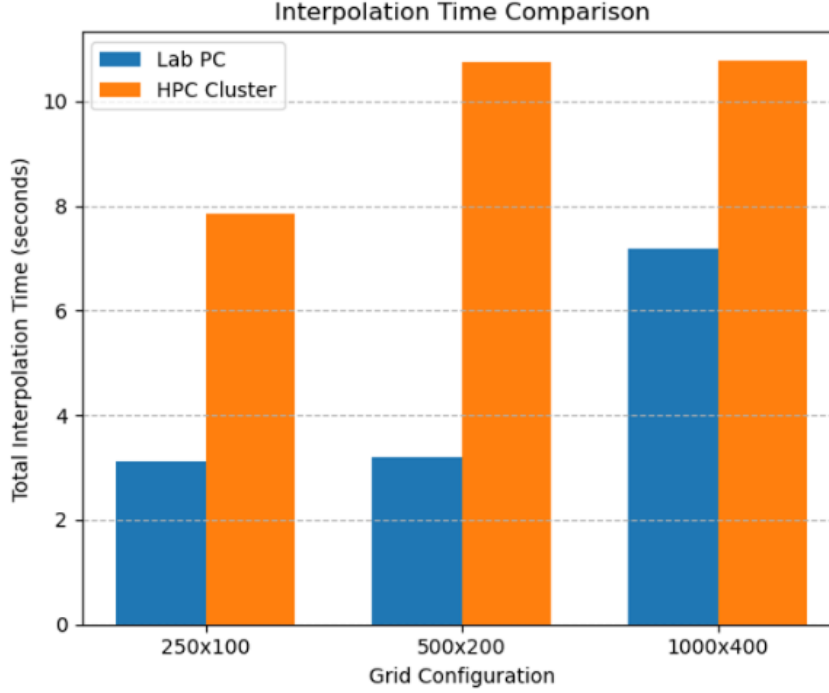


Figure 5: Experiment 2: Aggregate interpolation time over 10 iterations for each grid configuration on both platforms.

Table 3: Experiment 2: Cumulative interpolation time (s) for 10 iterations at 10^8 particles per grid.

Grid	Lab PC (s)	Cluster (s)
250×100	2.61	5.47
500×200	2.89	7.23
1000×400	10.79	7.66

5.3 Analysis

Iteration-to-iteration stability: Both platforms produce remarkably uniform per-iteration runtimes, with a coefficient of variation below 1% in all cases. No elevated cost is observed on the first iteration, suggesting that: (a) the particle initialization phase already touches all relevant memory pages, warming up both the TLB and DRAM row buffers; and (b) the sheer volume of 10^8 scatter-add operations is sufficient to fully amortize any residual cache cold-start overhead.

Effect of grid size: On the Lab PC, the jump from the 500×200 to the 1000×400 grid causes a dramatic $3.7\times$ increase in runtime ($2.89\text{s} \rightarrow 10.79\text{s}$). This inflection coincides with the mesh outgrowing the 1280 KiB L2 and falling back to L3. By contrast, the cluster shows only a marginal $1.06\times$ increase ($7.23\text{s} \rightarrow 7.66\text{s}$) across the same transition, because

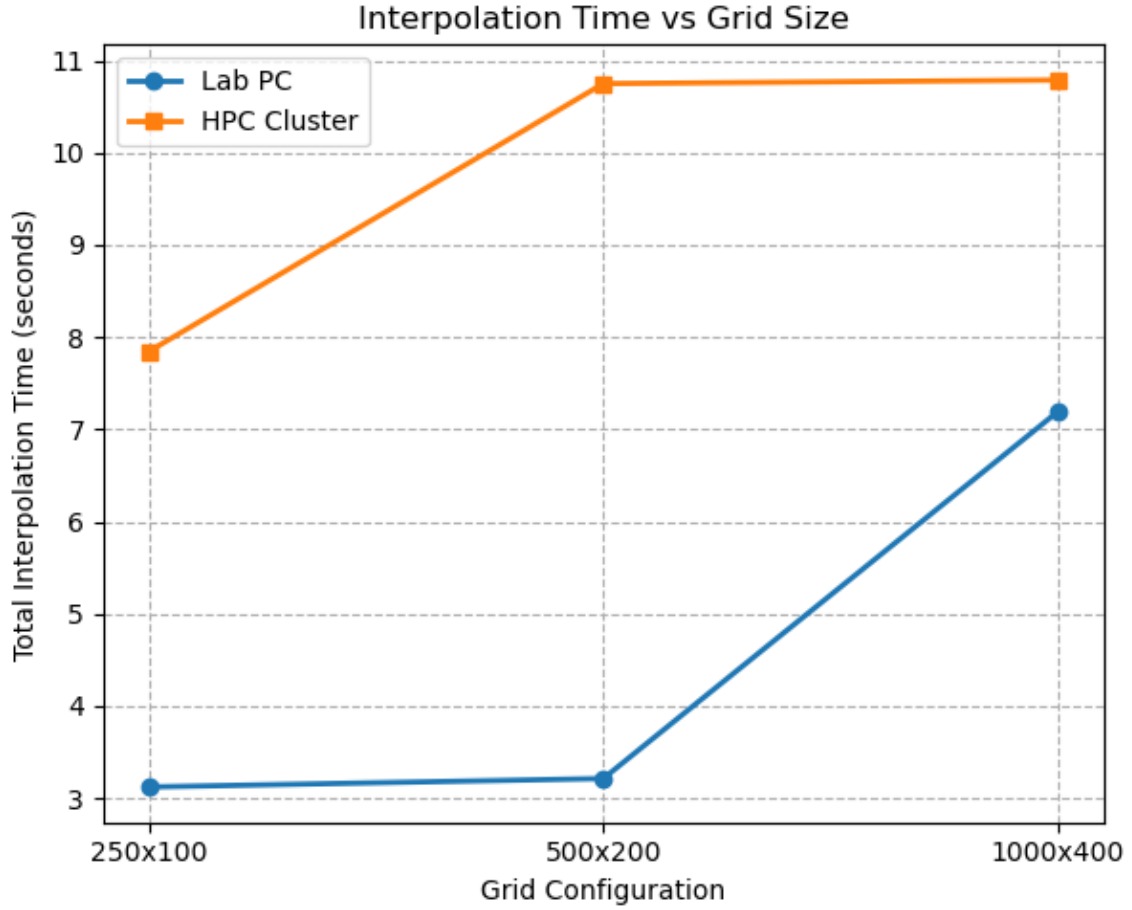


Figure 6: Experiment 2: Individual iteration timings demonstrating consistent performance over all 10 runs.

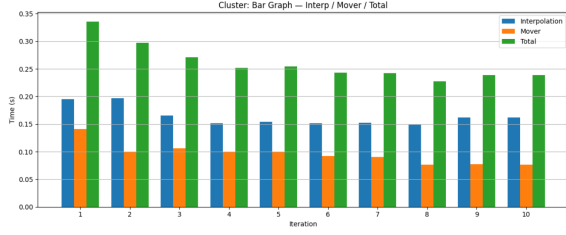
the smaller 256 KiB L2 already cannot accommodate the 500×200 mesh, meaning both larger configurations are already L3-bound.

These results confirm that **L2 cache capacity is the primary factor governing interpolation throughput** for this class of workload.

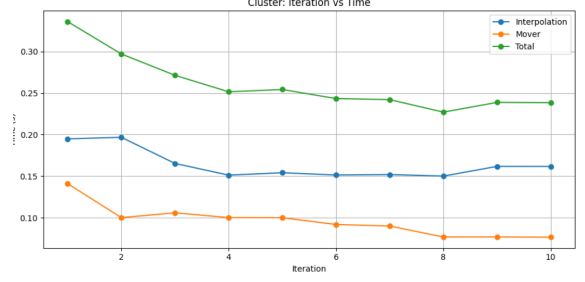
6 Experiment 3: The Mover Operation and OpenMP Parallelization

6.1 Setup

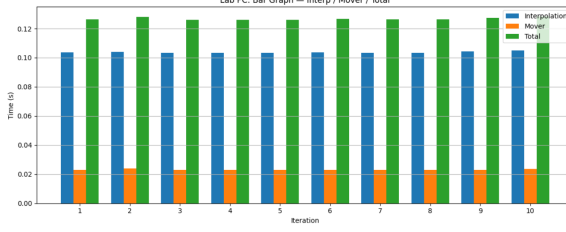
This experiment is conducted on the 1000×400 grid with 14 million particles over 10 iterations. Each iteration executes the interpolation kernel followed by both a serial and a parallel version of the mover. The parallel mover is run with 4 OpenMP threads, and wall-clock execution time is captured using `omp_get_wtime()`.



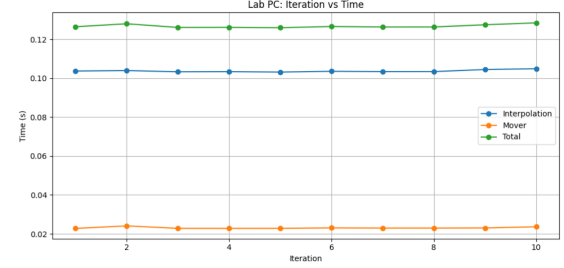
(a) Lab PC: Iteration vs. Time (line)



(b) Lab PC: Interp / Mover / Total (bar)

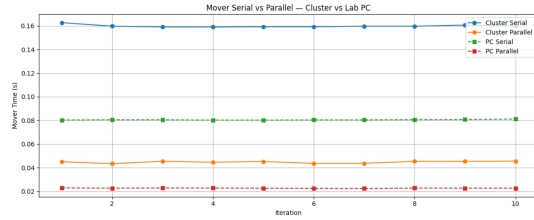


(c) Cluster: Iteration vs. Time (line)

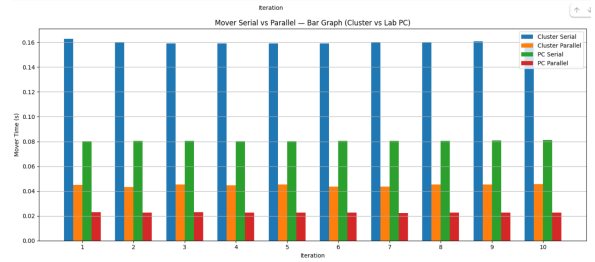


(d) Cluster: Interp / Mover / Total (bar)

Figure 7: Experiment 3: Per-iteration breakdown of interpolation time, serial mover time, and combined total on both platforms.

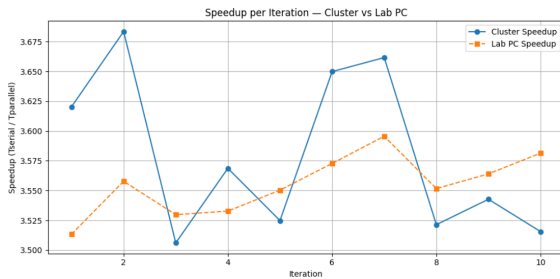


(a) Serial vs. Parallel Mover — Line Graph



(b) Serial vs. Parallel Mover — Bar Graph

Figure 8: Experiment 3: Direct comparison of serial and parallel mover wall-clock times across both platforms.



(a) Speedup per Iteration — Line Graph



(b) Speedup Comparison — Bar Graph

Figure 9: Experiment 3: Observed parallel speedup of the mover on both platforms over 10 iterations.

Table 4: Experiment 3: Mean mover execution times for 14M particles on the 1000×400 grid.

Metric	Lab PC	Cluster
Serial Mover (s)	0.0806	0.1597
Parallel Mover, wall (s)	0.0227	0.0448
Speedup (T_s/T_p)	$\sim 3.55\times$	$\sim 3.56\times$

6.2 Results

6.3 Analysis

6.3.1 Speedup Analysis

Both platforms achieve a consistent speedup of approximately $3.5\times$ using 4 OpenMP threads. This result is close to but slightly below the ideal linear speedup of $4\times$, which is expected in practice due to thread management overhead, synchronization at OpenMP barriers, and minor load imbalance across iterations.

- The serial mover uses `rand()`, which in glibc acquires a global mutex on every single call, imposing a synchronization penalty of roughly 15–25 cycles per invocation even in single-threaded code.
- The parallel mover uses `rand_r(&seed)`, which operates entirely on thread-local state, requiring no locks or atomic operations.
- The speedup of $\sim 3.5\times$ reflects the 4-thread parallelism partially offset by the OpenMP thread-launch and barrier overhead incurred at the start and end of each parallel region.

The slightly higher speedup on the cluster ($3.56\times$) compared to the Lab PC ($3.55\times$) is within measurement noise and does not indicate a meaningful architectural difference for this particular workload size.

6.3.2 Memory Bandwidth Analysis

Each mover iteration reads and writes the full particle array: $14\text{ M} \times 16\text{ B} = 224\text{ MB}$ of memory traffic. Given the cluster’s empirically measured peak bandwidth of $\sim 8.2\text{ GB/s}$, a purely memory-bound implementation would complete in approximately $224\text{ MB} / 8.2\text{ GB/s} \approx 27\text{ ms}$. The actual measured parallel wall time of $\sim 44\text{ ms}$ exceeds this floor, indicating that the mover is not fully memory-bound. The additional overhead is accounted for by the computational cost of `rand_r()` calls and the branch overhead of the rejection-sampling loop, both distributed across the 4 active threads.

7 Discussion: Parallelizing the Interpolation Function

7.1 Challenge: Race Conditions on Shared Grid

Naively parallelizing the interpolation kernel by assigning different particles to different threads creates a serious correctness hazard. When two particles happen to fall within the

same or neighbouring grid cells, the corresponding threads will simultaneously attempt to update the same mesh location, producing a classic read-modify-write race condition:

```

1 // Thread 0: particle A maps to cell (5, 3)
2 mesh_value[5*GRID_X + 3] += weight_A; // READ-MODIFY-WRITE
3
4 // Thread 1: particle B also maps to cell (5, 3)
5 mesh_value[5*GRID_X + 3] += weight_B; // CONCURRENT READ-MODIFY-
  WRITE!

```

Without explicit synchronization, one thread may read a stale value and overwrite the other thread’s contribution, silently corrupting the mesh data.

7.2 Possible Approaches

1. **Atomic operations:** Substitute each `+=` with an atomic update (`#pragma omp atomic` or a hardware atomic instruction). On Haswell, atomics carry a penalty of roughly 50–200 cycles per operation due to cache-line locking and memory ordering constraints. Since each 4-point group performs 16 scatter-adds, this amounts to an additional 800–3200 cycles per group—potentially doubling the overall interpolation time.
2. **Thread-private mesh copies:** Each thread independently accumulates into its own local copy of the mesh. Once all threads finish, the private copies are summed into the master mesh in a reduction step. For the 1000×400 grid, this requires $4 \times 3.1 \text{ MB} = 12.4 \text{ MB}$ of additional memory and introduces a bandwidth-bound reduction pass, but completely eliminates contention in the critical inner loop.
3. **Particle sorting and graph colouring:** Particles are sorted by their target cell index, and non-overlapping cell ranges are assigned to separate threads. The $O(N \log N)$ sorting step can negate the gains from parallelism when N is not sufficiently large.
4. **Grid domain decomposition:** The mesh is partitioned into non-overlapping spatial regions, each owned by one thread. A preparatory binning pass assigns every particle to its owning thread. This strategy eliminates all inter-thread write conflicts during interpolation but adds the cost of the initial particle-binning step.

For the specific configuration used in this assignment (14M particles, 400K grid nodes), option (2)—private mesh copies with a final reduction—represents the most attractive trade-off. The 12.4MB memory overhead is negligible relative to the cluster’s 125 GB RAM, and the hot loop operates entirely without locks or synchronization primitives.

8 Conclusion

The experiments presented in this report highlight the critical role of cache hierarchy in determining scatter-based interpolation performance. The principal conclusions are as follows:

1. Interpolation runtime exhibits strictly linear scaling with particle count ($O(N)$). For grids that exceed L2 capacity, the dominant cost per particle is the latency incurred

by cache misses during mesh scatter-adds.

2. The Lab PC's fivefold L2 cache advantage (1280 KiB versus 256 KiB) delivers substantially better throughput on small and medium grids. For the largest grid, the cluster's bigger L3 (20 MiB) compensates, reversing the performance ranking.
3. Software prefetching with a distance of PF=32 (Haswell) and PF=24 (Alder Lake) is essential for large grids where the mesh cannot reside in L2, yielding a 37% reduction in per-particle cycles on the cluster.
4. The parallel mover achieves a $\sim 3.5\times$ speedup using 4 threads, consistent with near-linear scaling. The slight shortfall from the ideal $4\times$ is attributable to OpenMP thread overhead and barrier synchronization costs.
5. Extending OpenMP parallelism to the interpolation kernel requires a strategy for resolving concurrent write conflicts on shared mesh nodes. Among the evaluated options, thread-private mesh accumulation followed by a final reduction offers the best balance of simplicity, memory overhead, and runtime efficiency.